

Shape Abstraction via Marching Differentiable Support Functions

Sunkyung Park, Jeongmin Lee, Dongjun Lee*

Seoul National University

{sunk1136, ljm1gh, djlee}@snu.ac.kr

Abstract

Shape abstraction, simplifying shape representation into a set of primitives, is a fundamental topic in computer vision. The choice of primitives shapes the structure of world understanding, yet achieving both high abstraction accuracy and versatility remains challenging. In this paper, we introduce a novel framework for shape abstraction utilizing a differentiable support function (DSF), which offers unique advantages in representing a wide range of convex shapes with fewer parameters, providing smooth surface approximation and enabling differentiable contact features (gap, point, normal) essential for downstream applications involving contact-related problems. To tackle the associated optimization and combinatorial challenges, we introduce two techniques: differentiable shape parameterization and hyperplane-based marching to enhance accuracy and reduce DSF requirements. We validate our method through experiments demonstrating superior accuracy and efficiency, and showcase its applicability in tasks requiring differentiable contact information.

1. Introduction

Shape representation is pivotal in computer vision and serves as a fundamental building block for explaining and understanding the world in a computable form. Representation techniques range from classical models like meshes and grids to more recently emerging methods such as implicit fields and encoders [37, 46], each selected for their suitability to specific applications. Meanwhile, shape abstraction simplifies these representations into more manageable primitives, facilitating a more structured understanding of the world. This is also known to align with how humans perceive and understand scenes by separating them into components [2], and often enables more efficient processing across a variety of applications, including semantic labeling [49], editing [29], and physics simulation [8].

In this regard, shape abstraction has been explored

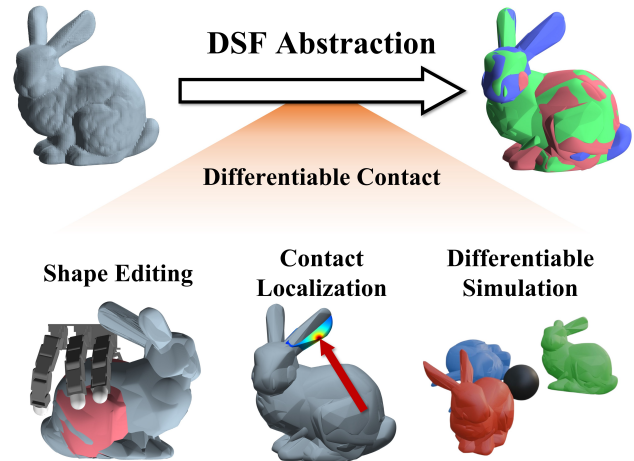


Figure 1. Our method abstracts an object represented by SDF into DSFs. The abstracted results can be utilized in differentiable formulations of shape editing, contact localization, and differentiable simulation. For shape editing, the abstracted DSFs are modified by contact information.

through various research efforts in recent years, utilizing different levels of primitives including planar sections [26, 27], cuboids [38, 40, 48], polytopes [5, 8], and, more commonly, superquadrics [22, 23, 29, 35]. In this paper, we aim to establish a novel framework for shape abstraction based on a new type of primitive—differentiable support function (DSF [19]), a recently proposed technique for representing convex geometry. We suggest that DSF offers the following notable advantages in shape abstraction: 1) unlike conventional primitives, it can accommodate a wide range of convex shapes and, compared to more complex polytopes or meshes, requires fewer parameters to accurately represent smooth surfaces; and 2) the abstraction introduces a novel utility—differentiable contact, where contact features (gap, point, normal) between shapes are guaranteed to be differentiable with respect to both shape parameters and object pose. This allows the abstraction to be utilized in contact-related problems, including contact localization, differentiable simulation, and shape post-optimization by leveraging additional geometric and physical information.

*Corresponding author

Our DSF-based shape abstraction is performed by extracting a set of DSF parameters from signed distance function (SDF) data, which is a fundamental low-level volumetric information format that can be readily transformed from raw sensory inputs [6, 32, 34]. However, this extraction process can involve complex optimization and combinatorial challenges, as also pointed out in other works on computational method for primitive abstraction [22, 44]. To address these issues, we propose two techniques tailored for DSF-based shape abstraction: 1) differentiable shape parameterization, which enables efficient and robust fitting of DSF into the desired geometry using unconstrained optimization; and 2) hyperplane-based marching, which enhances abstraction accuracy for arbitrary complex shapes and reduces the number of required DSFs. We demonstrate the advantages of our abstraction scheme in terms of both accuracy and efficiency through various experiments, and also provide examples of downstream tasks that can be performed using the differentiable contact utility.

2. Related Works

2.1. Geometry Representation and Abstraction

In computer vision and graphics, meshes are a standard geometry representation. Various methods have been proposed to generate meshes from sensory data, such as RGB images and point clouds, using neural networks [5, 13, 17, 45]. Despite their versatility, obtaining high-quality meshes is challenging, and representing smooth surfaces requires a large number of vertices. Another widely used representation is implicit functions, studied in various contexts based on data sources and applications [4, 20, 21, 28, 32, 34]. While effective for representing continuous objects in a scalable manner, they are difficult to structurally interpret and usability is often limited by their output type.

Abstracting geometry into convex components has been researched in the convex decomposition field. Mesh decomposition [25, 47], which decomposes a mesh into convex hulls, is widely utilized in physics simulations. Neural network-based methods have been developed to generate convexes based on various primitives [8, 11, 35, 38, 40, 48], though these methods face challenges in terms of generalizability and accuracy. More recently, computational methods using superquadrics have been proposed [22, 23], demonstrating superior representation performance compared to neural networks. These methods progressively extract primitives from objects, and the number of components varies depending on the shape of the object. While superquadrics can generalize to shapes beyond cuboids and ellipsoids, they remain restricted to symmetric geometries. The DSF introduced in [19], on the other hand, can represent a broader range of smooth convex shapes than superquadrics, while the abstraction method has not been proposed.

2.2. Contact-Related Problems

Contact-related problems typically arise in fields such as physics simulation and robotics. These problems are inherently linked to the shape representation of objects, with computation of contact features between them. While contact computation have been extensively studied for convex shapes [1, 12, 19, 43], performing such queries for general implicit functions is often challenging due to incompatibility in output values [8]. As a result, only a limited set of shape pairs, such as sphere-sphere, convex-convex, and SDF-point, have been practically employed for contact-related problems. From a computer vision perspective, this connects with various shape reconstruction and abstraction algorithms. For instance, voxel and SDF reconstruction methods have been widely used in tasks like grasping [41, 42] and collision avoidance planning [28, 31]. Also several primitive-based abstraction for sensory inputs [8, 29] has been applied to physics simulations.

Recently, leveraging gradients in contact-related problems is gaining attention to improve computational efficiency, enabling advancements in areas such as reinforcement learning [9], manipulation planning [33], and system identification [10, 14]. However, querying differentiable contact between shapes remains an ongoing research. Studies such as [14, 39] focus on basic geometries like spheres, cones, and boxes. Approximate gradients using sampling [18] or randomized smoothing [30] have also been proposed, with natural connections to 3D reconstruction techniques in computer vision. In contrast, our work aims to abstract shapes by leveraging DSF, which is a theoretically guaranteed shape class that provides analytic gradients.

3. Differentiable Shape Parameterization

3.1. DSF Parameters

The notion of differentiable support function (DSF) utilized in this work is primarily based on the geometry representation presented in [19]. This representation leverages the theoretical property that the following differentiable convex function serves to a support function for a specific convex geometry:

$$h(x) = \left(\sum_{i=1}^{N_v} \{ \max(v_i^T x, 0) \}^p \right)^{\frac{1}{p}} \quad (1)$$

where $x \in \mathbb{R}^3$ denotes the given direction, $v_i \in \mathbb{R}^3$ is a vertex, $p \in \mathbb{R}$ is a smoothness parameter, and N_v is the number of vertices. From the definition of the support function, a support point is obtained as the partial derivative of $h(x)$ with respect to x .

In [19], to ensure a valid geometry, the vertices in Eq. (1) are designed so that the origin is located within their convex hull. In this paper, we extend this approach by introducing

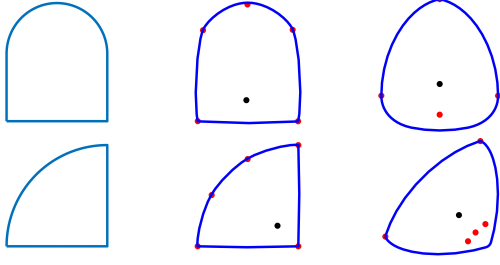


Figure 2. DSF fitting results with α (middle) and without α (right, where $\forall \alpha_i$ is fixed at 0 for geometric center) for the given convex shapes (left). Black and red dots represent the center and vertices of DSF, respectively.

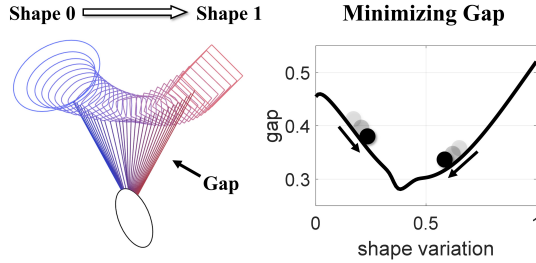


Figure 3. A simple example of the shape optimization problem utilizing differentiable contact. Contact feature (gap) between two DSFs is computed along a smooth variation of shape parameter ϕ (left). Since the gap is differentiable with respect to ϕ , it can be minimized using gradient-based solvers (right).

a center variable $c \in \mathbb{R}^3$, allowing the vertices to be defined relative to an arbitrary center position as follows:

$$h(x) = \left(\sum_{i=1}^{N_v} \left\{ \max \left((v_i - c)^T x, 0 \right) \right\}^p \right)^{\frac{1}{p}}. \quad (2)$$

In Eq. (2), the center c still must be located within the convex hull of v . To incorporate this constraint in a differentiable manner, we propose the following parameterization:

$$c = \sum_{i=1}^{N_v} v_i \beta_i \quad \text{where} \quad \beta_i = \frac{e^{\alpha_i}}{a \sum_{j=1}^{N_v} e^{\alpha_j}} + \frac{a-1}{aN_v} \quad (3)$$

which is designed to enforce $\sum_{i=1}^{N_v} \beta_i = 1$ and $\forall \beta_i \in [\frac{a-1}{aN_v}, \frac{a+N_v-1}{aN_v}]$ with $a > 1$. Through Eq. (3), the center c is parameterized by unbounded $\alpha_i \in \mathbb{R}$. Similarly, the smoothness parameter p is modeled using the unbounded shape parameters $\tilde{p}$ as follows:

$$p = (\tanh \tilde{p} + 1) \frac{p_{\max} - 2}{2} + 2 \quad (4)$$

which is designed to ensure $p \in (2, p_{\max})$ to maintain differentiability of (1) and avoid excessive exponentiation.

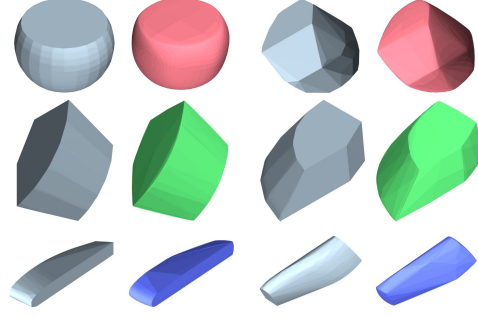


Figure 4. DSF fitting results (colored) for the given polygonal meshes (gray). Despite using only 20 vertices, the fit remains robust across a wide range of geometries.

As a result, the DSF is defined through the shape parameters $\phi = (v_{1:N_v}, \alpha_{1:N_v}, \tilde{p}) \in \mathbb{R}^{4N_v+1}$. The proposed parameterization offers two main advantages. First, we can achieve high representation performance with a few vertices v by flexibly adjusting c based on the interpolation parameters introduced in Eq. (3). This is especially useful for shapes with both curved and flat surfaces (see Fig. 2 for examples). Second, the support function $h(x, \phi)$ and also corresponding support point are differentiable with respect to the shape parameters ϕ , enabling its effective use in the DSF fitting process explained in Sec. 3.2. This further implies that contact features between DSFs are differentiable with respect to ϕ (see supplementary material for more details), allowing for its use in shape optimization for specific downstream tasks (see Fig. 3 for an example). Note that this differentiable contact is not naturally achievable when using polytopes or superquadrics as primitives.

3.2. DSF Fitting

Based on the shape parameterization in Sec. 3.1, we can formulate the problem of fitting a DSF to an arbitrary convex geometry as the following optimization:

$$\min_{\phi \in \mathbb{R}^{4N_v+1}} \sum_{i=1}^{N_h} \left(h(x_i, \phi) + c^T x_i - \hat{h}_i \right)^2 \quad (5)$$

where $\hat{h}_i$ is the support function value in the direction x_i for the target convex geometry, and N_h is the number of direction samples.

From our differentiable shape parameterization, the solution ϕ^* of Eq. (5) can be obtained by standard nonlinear least square solvers. However, in practice, using an integer p significantly reduces the computational cost of exponentiation in Eq. (2) compared to a continuous p . Thus, an integer p close to the optimal p^* is selected, and Eq. (5) is re-solved with fixed p . The results of fitting are illustrated in Fig. 4. The arbitrary convex shapes in Fig. 4 are accurately fitted using only 20 vertices, demonstrating the effectiveness of our DSF parameterization and fitting formulation.

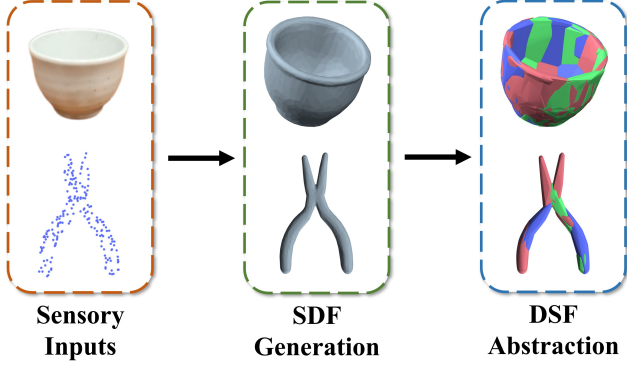


Figure 5. The abstraction pipeline for sensory inputs. The SDF can be generated using computational or learning-based algorithms proposed in various studies. Our algorithm abstracts DSF components from the SDF data.

4. Hyperplane-based Marching

In this section, we propose a method to abstract arbitrary complex shapes as a set of DSFs. As illustrated in the pipeline in Fig. 5, we first generate the SDF from the sensory inputs, using computational or learning-based methods proposed in various studies [20, 21, 32, 34]. Consequently, we can obtain a set of SDF data: point (P), corresponding SDF value (S), and gradient of SDF (G).

One naive approach to generating DSFs using these SDF data is to extract a mesh using the marching cube algorithm [24], decompose it into convex meshes [25, 47], and then fit the DSFs. However, the fitting error from Eq. (5) may accumulate throughout the entire process. Furthermore, this approach may yield an excessive number of convex components since certain geometries that are misaligned with the principal axis [36] cannot be effectively decomposed by axis-aligned cutting planes [47]. In contrast, the *marching scheme*, iteratively generating the primitives within the object, can effectively handle the accumulation of fitting error and can intuitively abstract the misaligned geometries. Therefore, we design a marching method for DSF to reduce the number of convex components while improving the representation performance.

A recent study [23] aligns with our problem of interest, proposing a method for marching superquadrics on a given truncated SDF. This approach gradually identifies a collection of voxelized distances and extracts primitives from them. However, such voxel-based marching is unsuitable for our framework, as distance data is not readily computable from the DSF. Instead, we introduce a hyperplane-based marching approach, which identifies a collection of hyperplanes and fits the DSF within the convex region (intersection of halfspaces) based on the formulation described in Eq. (5), progressively filling the occupied space. To

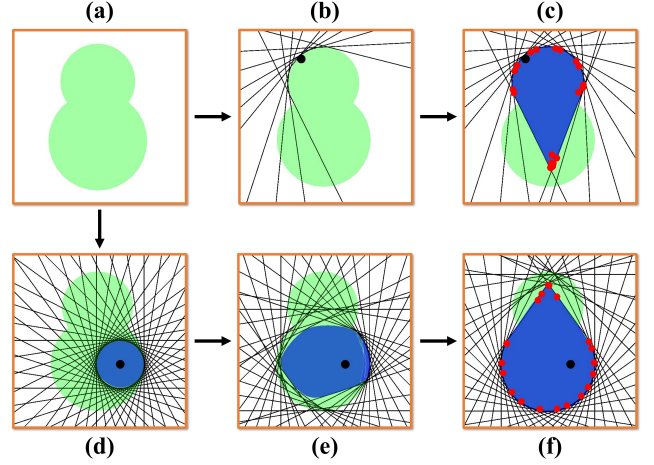


Figure 6. Visualization of the region reduction (b-c) and expansion (d-f) process for the input geometry (a). Hyperplanes and the convex region are visualized by straight lines and blue regions. Black dots are the p_m for each method and red dots are the vertices of DSF fitted to the convex region.

obtain a sufficiently large convex region, we propose two methods: the *region reduction* and the *region expansion*, which are visualized in Fig. 6.

4.1. Hyperplanes

We introduce several notations for the hyperplanes used in our study. A hyperplane, defined by an equation $n^T x - o = 0$, is denoted as $H = [n; o] \in \mathbb{R}^4$ where n and o are a unit vector in $\mathbb{R}^3$ and a scalar offset, respectively. Then we define the minimum distance from a point $u \in \mathbb{R}^3$ to a set of hyperplanes $\mathcal{H}$ as follows:

$$d(\mathcal{H}, u) = \max_{H_i \in \mathcal{H}} (n_i^T u - o_i). \quad (6)$$

Similarly, the hyperplane corresponding to the minimum distance from u is determined as follows:

$$H(\mathcal{H}, u) = \operatorname{argmax}_{H_i \in \mathcal{H}} (n_i^T u - o_i). \quad (7)$$

4.2. Region Reduction

The region reduction method is inspired by the algorithm for generating obstacle-free space [7], sequentially slicing the region with hyperplanes. This method is initialized at a point $p_m \in P$ randomly selected where $s_m < -\epsilon$ for $s_m \in S$ and a small $\epsilon > 0$. Using the closest point $p_j \in P$ from p_m where $s_j \in S$ and $s_j > 0$, along with its gradient $g_j \in G$, a normal of a hyperplane is defined as follows:

$$n_j = \begin{cases} g_j & \text{if } g_j^T (p_j - p_m) - s_j > 0 \\ \frac{p_j - p_m}{\|p_j - p_m\|} & \text{otherwise} \end{cases}. \quad (8)$$

Algorithm 1: Region Reduction

```

1 Input:  $P, S, G, \mathcal{D}$ 
2 Initialize  $\mathcal{H} = \emptyset$ 
3 Select  $p_m \in P$ 
4  $\mathcal{H} \leftarrow \mathcal{H} \cup H_{D'}$  for all  $D' \in \mathcal{D}$ 
5  $R = \text{intersection}(\mathcal{H})$ 
6 while  $\exists p_i \in R$  s.t.  $s_i > 0$  do
7    $p_j \leftarrow$  the closest point in  $R$  to  $p_m$  with  $s_j > 0$ 
8   Calculate  $n_j$  by using Eq. (8)
9    $\mathcal{H} \leftarrow \mathcal{H} \cup H_j$  where  $H_j$  as in Eq. (9)
10   $R = \text{intersection}(\mathcal{H})$ 
11 end
12 Fit DSF  $D$  to  $R$  by using Eq. (5)
13 Output:  $D$ 

```

Consequently, a hyperplane H_j is defined as follows:

$$H_j = [n_j; n_j^T(p_j - p_m) - s_j]. \quad (9)$$

Using Eq. (9), we can eliminate the points $p_i \in P$ outside of H_j , and the next closest point p_j from p_m is determined for the next H_j until there is no remaining p_j with $s_j > 0$. Through iterations, this process results in a convex region R , and a DSF is fitted to the hyperplanes sampled from R by solving Eq. (5).

After each fitting, all points $p \in P$ with $s < 0$ inside the generated DSF are excluded to prevent the creation of identical DSFs. For any subsequent region reduction process, additional hyperplanes are employed to control overlap with existing DSFs, defined as

$$H_D = [\delta_D; \|\delta_D\| + o_d] \quad (10)$$

where δ_D is the minimum distance vector from p_m to the DSF, computed by the minimum distance solver in [1], D and o_d is the overlap parameter that determines the amount of allowed overlap. In practice, assigning a proper positive value to o_d helps prevent discontinuous gaps between DSFs and reduces the overall number of required DSFs (see also Sec. 5.2). Overall process is summarized in Alg. 1.

4.3. Region Expansion

In this method, a convex region defined by the intersection of halfspaces is incrementally expanded using a collection of sampled distances. The halfspaces are defined by a set of hyperplanes $\mathcal{H}$ with fixed normals n_j , and the hyperplane offsets are adjusted based on $d(\mathcal{H}, p)$, where p is sampled with a small value of $|d(\mathcal{H}, p)|$. This method is initialized at the point $p_m \in P$ for minimum SDF value $s_m \in S$, setting each initial o_j of $H_j \in \mathcal{H}$ to $-s_m + n_j^T p_m$: a sphere centered at p_m . For each expansion, a set of points $P_s \subseteq P$ is sampled as

$$P_s = \{p_i \in P \mid |d(\mathcal{H}, p_i)| < t\} \quad (11)$$

Algorithm 2: Region Expansion

```

1 Input:  $P, S$ 
2 Select  $p_m \in P$ 
3 Initialize  $\mathcal{H} = \{H_j = [n_j; -s_m + n_j^T p_m]\}_{j=1}^{N_d}$ 
4 while offsets of  $\mathcal{H}$  do not converge do
5   Sample  $P_s$  by using Eq. (11)
6   for  $j = 1, 2, \dots, N_d$  do
7     Define  $P_{s,j}$  by using Eq. (12) and  $P_s$ 
8     Define  $Y_j$  by using Eq. (13) and  $P_{s,j}$ 
9     Compute  $o_j^*$  by using Eq. (15)
10     $H_j \leftarrow [n_j; o_j^*]$ 
11   end
12 end
13  $R = \text{intersection}(\mathcal{H})$ 
14 Fit DSF  $D$  to  $R$  by using Eq. (5)
15 Output:  $D$ 

```

where $t \in \mathbb{R}$ is a truncation value. For each $p_i \in P_s$, the minimum distance and corresponding hyperplane H_j are computed by Eq. (6) and Eq. (7). Conversely, a set of points corresponding to each H_j is defined as follows:

$$P_{s,j} = \{p_i \in P_s \mid H(\mathcal{H}, p_i) = H_j\}. \quad (12)$$

To expand H_j for a single point $p_i \in P_{s,j}$, the offset of H_j must be added by $d(\mathcal{H}, p_i) - s_i$. Generalizing this to multiple points, a set of distances to be added to the offset of H_j is defined as follows:

$$Y_j = \{d(\mathcal{H}, p_i) - s_i \mid p_i \in P_{s,j}\}. \quad (13)$$

Using Y_j corresponding to H_j , the optimization problem for the offset of H_j is formulated as

$$y^* = \underset{y \in \mathbb{R}}{\operatorname{argmin}} \sum_{k=1}^{N_j} w_k (y - y_k)^2 \quad (14)$$

where $N_j = |Y_j|$, $y_k \in Y_j$, and w_k is a weight for y_k , which is 1 for $s_k \geq 0$ and 10^{-4} for $s_k < 0$. The optimal offset o_j^* for H_j is determined by the closed-form solution of Eq. (14) as follows:

$$o_j^* = o_j + \frac{\sum_{k=1}^{N_j} w_k y_k}{\sum_{k=1}^{N_j} w_k}. \quad (15)$$

By iterating Eqs. (11) to (15), $\mathcal{H}$ can be fitted to P_s and expanded to a larger region, neglecting minor concavities as the solution o_j^* in Eq. (15) is based on the weighted average distance for $P_{s,j}$. Once all the o_j converge, a convex region R is obtained by the intersection of halfspaces defined by $\mathcal{H}$. Then, a DSF is fitted to the hyperplanes sampled from R , and $p_i \in P$ inside the generated DSF are just removed from the algorithm to prevent the generation of identical DSFs. The details of this method are provided in Alg. 2.

Algorithm 3: DSF Abstraction

```
1 Input:  $P, S, G$ 
2 Initialize  $\mathcal{D} = \emptyset$ 
3 while  $\min_{s \in S} s \leq t_{end}$  do
4   if  $\min_{s \in S} s \leq t_s$  then
5      $D \leftarrow \text{RegionReduction}(P, S, G, \mathcal{D})$ 
6   else
7      $D \leftarrow \text{RegionExpansion}(P, S)$ 
8   end
9   if  $D$  is not degenerated then
10     $\mathcal{D} \leftarrow \mathcal{D} \cup D$ 
11  end
12 end
13 Output:  $\mathcal{D}$ 
```

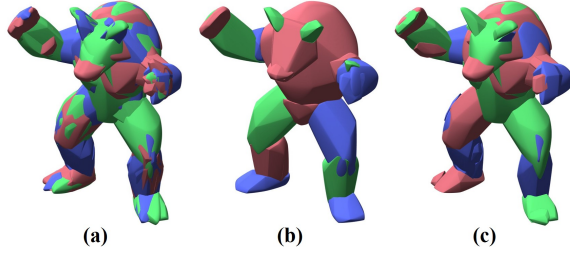


Figure 7. Comparison of abstraction methods. The object is the armadillo from the Stanford 3D scanning repository [6]. (a) Region reduction method (49 DSFs). (b) Region expansion method (22 DSFs). (c) Hybrid method (31 DSFs).

4.4. Hybrid Method and Self-Degeneration

Although the region reduction method has a high representation performance, its strict exclusion of exterior regions may result in an excessive number of DSFs for highly concave geometries. While the region expansion method can neglect minor concavities based on the weighted average distance formulation in Eq. (15), reducing the number of DSFs for highly concave geometries, it may also result in a loss of surface detail. To address these limitations, we propose a hybrid method that balances representation performance with the number of convex components.

Initially, Alg. 1 is used to produce DSFs with high representation power. If the minimum SDF value exceeds a threshold t_s , Alg. 2 is applied to efficiently cover the remaining parts. The details of the hybrid method are described in Alg. 3. The comparison among the three methods is visualized in Fig. 7. After fitting each DSF to R , DSFs with small volumes or significant discrepancies from the SDF are degenerated. The details of the degeneration process are provided in the supplementary material.

5. Experiments

5.1. Abstraction Results

Our DSF abstraction method using hyperplane-based marching (MDSF, Alg. 3) is compared with three different baselines: an optimization method based on a neural network, a fitting method, and a marching method. The optimization method based on a neural network is implemented by CvxNet [8] trained for a single object, optimizing all primitives simultaneously. The fitting method is implemented by the state-of-the-art mesh decomposition algorithm (CoACD [47]), and by fitting the DSF to each convex mesh from CoACD (CoDSF). For the marching method, the state-of-the-art marching algorithm for superquadrics (MP [23]) is compared with our method.

For CvxNet [8], the batch size and the maximum training steps are set to 32 and 5000, respectively. For CoDSF and our method, we set N_v in Eq. (2) and N_h in Eq. (5) to 20 and 1742, respectively. The overlap parameter o_d in Alg. 1 is set to a large value, greater than the object size, to allow intersection between DSFs.

The evaluation is conducted on the YCB dataset [3] and the robot links CAD data provided by the MuJoCo Menagerie [50], including 5 robots with a total of 51 links and 14 objects from YCB. The comparison metrics are intersection over union (IoU), Chamfer-L1 distance, and the number of primitives (N_c). Definitions for IoU and Chamfer-L1 distance are provided in the supplementary material. IoU for DSF is calculated by determining occupancy for each point sample using the minimum distance solver for DSF [1]. For Chamfer-L1 distance, the surface points of superquadrics are sampled via the equal-distance sampling method in [22], and the surface points of DSF are obtained from a fine mesh derived from the DSF.

The abstraction results are visualized in Fig. 8. The results of IoU, Chamfer-L1, and N_c are summarized in Tab. 1. Our DSF abstraction produces higher IoU, lower Chamfer-L1, and lower N_c than all other baselines, which demonstrates abstraction efficiency: higher representation power with fewer primitives. The visualizations for the YCB dataset [3] are provided in the supplementary material.

5.2. Ablation Studies

In Tab. 2, our hybrid method is compared with the region reduction method and the region expansion method. All methods are evaluated on the same dataset and metrics as Sec. 5.1. The region reduction method produces high representation performance with large N_c while the region expansion method produces low representation performance with small N_c . Notably, all three methods produce smaller N_c than all other baselines in Tab. 1.

By setting o_d to be larger than the object size as in Sec. 5.1, the intersection between DSFs is perfectly al-

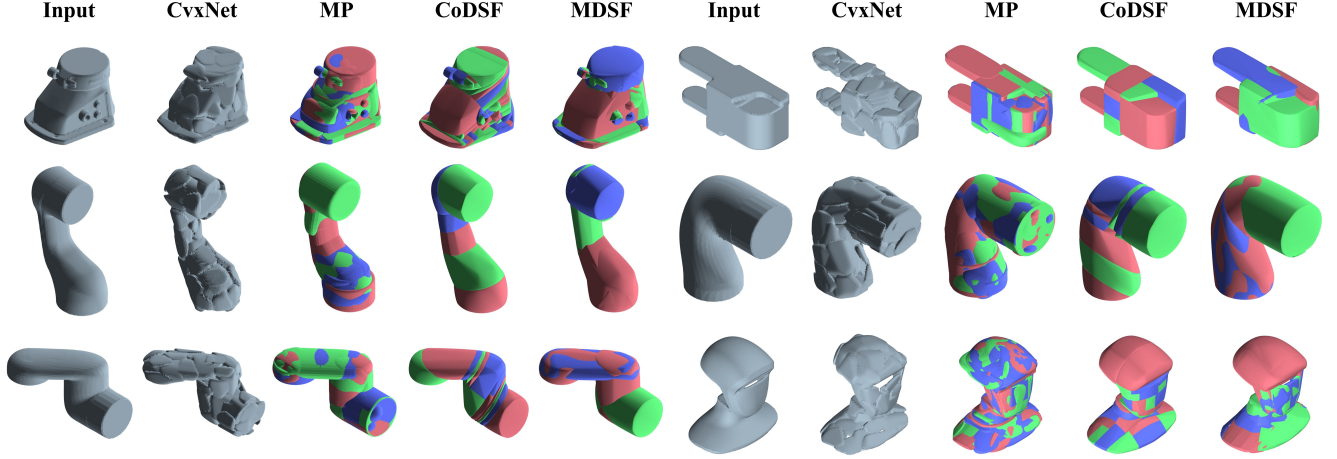


Figure 8. Abstraction results for the links of robots. From left to right: Input mesh, CvxNet [8], Marching-Primitives (MP [23]), DSF fitting for CoACD [47] results (CoDSF), and our DSF abstraction algorithm (MDSF, Alg. 3).

Category	IoU $\uparrow$				Chamfer-L1 $\downarrow$				$N_c \downarrow$			
	CvxNet [8]	MP [23]	CoDSF	MDSF	CvxNet [8]	MP [23]	CoDSF	MDSF	CvxNet [8]	MP [23]	CoDSF	MDSF
Franka-fr3	0.8772	0.9442	0.9510	0.9517	0.1760	0.0966	0.0866	0.0847	50	34.125	21.375	13
Lite6	0.8667	0.9226	0.9520	0.9533	0.2021	0.1363	0.0935	0.0919	50	37.7143	9.1429	7.2857
Xarm7	0.8551	0.9438	0.9488	0.9535	0.2056	0.0968	0.0901	0.0863	50	39.625	18.125	14.25
A1	0.7595	0.9221	0.9291	0.9424	0.3042	0.0784	0.0704	0.0650	50	21.8	8.8	9.8
G1	0.8154	0.9092	0.9293	0.9276	0.1605	0.0984	0.0812	0.0809	50	41.0870	25.9130	15.2174
YCB	0.7297	0.8949	0.9018	0.9076	0.1907	0.0904	0.0815	0.0764	50	38.7143	23.8571	16.5714
mean	0.8107	0.9171	0.9309	0.9333	0.1900	0.0988	0.0835	0.0810	50	37.6923	20.8308	13.8462

Table 1. Results of CvxNet [8], Marching-Primitives (MP [23]), DSF fitting for CoACD [47] (CoDSF), and our DSF abstraction algorithm (MDSF, Alg. 3). Chamfer-L1 is multiplied by 10. The number of objects for each category is 8, 7, 8, 5, 23, and 14, from top to bottom.

Method	IoU $\uparrow$	Chamfer-L1 $\downarrow$	$N_c \downarrow$
Red	0.9399	0.0780	17.3846
Exp	0.9170	0.0938	12.1077
Hyb	0.9333	0.0810	13.8462
Small- o_d	0.9399	0.0783	19.4308

Table 2. Ablation study. Red: the region reduction method (Alg. 1). Exp: the region expansion method (Alg. 2). Hyb: the hybrid method (Alg. 3). Small- o_d : the hybrid method with a small o_d . Hyb and Red use the same large o_d as in Sec. 5.1.

lowed, significantly reducing the overall number of required DSFs. In Tab. 2, the result with the large o_d (Hyb) is compared with the result with a small o_d (0.012 for normalized 3D space), denoted as Small- o_d . Although the hybrid method with the small o_d achieves higher IoU and lower Chamfer-L1 distance compared to the large o_d , it also results in a larger N_c , indicating that surface details are more accurately represented with an increased number of components. However, since N_c in both methods remains lower than the other baselines in Tab. 1, the o_d can be adjusted based on the task requirements, depending on whether intersection or overlap between DSFs is desirable.

6. Applications

6.1. Shape Editing

Since the gap between DSFs is differentiable with respect to ϕ , the abstracted DSFs can be edited to align with contact information. This problem is formulated as minimizing the gap between the abstracted DSF and the contact geometries (DSFs) while maintaining the original shape:

$$\min_{\phi \in \mathbb{R}^{4N_v+1}} \sum_{i=1}^{N_h} \left(h(x_i, \phi) + c^T x_i - \hat{h}_i \right)^2 + w_g \sum_{i=1}^{N_{con}} \delta_i^2 \quad (16)$$

where the first term is the same as Eq. (5), N_{con} is the number of contacts, w_g is a weight for the gap, and δ_i is a gap between the DSF and i -th contact geometry. Visualization for the shape editing process is provided in Fig. 9.

6.2. Contact Localization

Contact localization, identifying a contact point or detecting a link in collision, is essential for safety control and human-robot interaction [15, 16, 51]. Typically, this process relies on robot dynamics or neural networks using sensory data. Utilizing differentiable contact features with respect

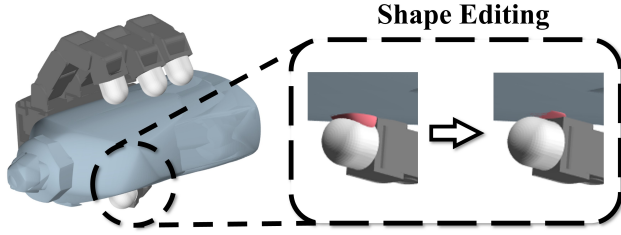


Figure 9. Illustrative example of shape editing using contact information with a gripper. The edited DSF is highlighted in red.

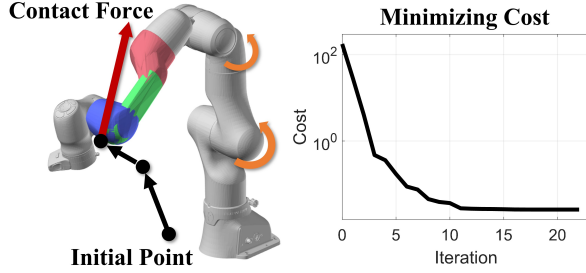


Figure 10. Visualization of contact localization problem. The initial point is optimized to lie on the surface of the DSF: a contact point generating a specified force and torque while meeting friction constraints of contact force.

to pose, computed by the contact feature solver [19] with our abstracted DSFs, this problem can be formulated as

$$\min_{p, \lambda \in \mathbb{R}^3} \|\lambda - \lambda_{proj}\|^2 + \|\gamma - J_c^T \lambda\|^2 + \delta^2 \quad (17)$$

where δ is a gap between a point p and DSF, λ is a contact force, λ_{proj} is a projection of λ on a friction cone defined by the contact point and normal computed between p and DSF, γ is a measurement of force and torque, and J_c is a contact Jacobian derived from the contact point. The DSF used in Eq. (17) is determined by the minimum gap between p for each iteration. The first term in Eq. (17) is a relaxation of friction constraint of contact force, and the second term is designed to match the contact force with sensory data. The result of Eq. (17) is illustrated in Fig. 10. The initial point in Fig. 10 is effectively optimized to the surface of DSF producing the measured force and torque γ .

6.3. Differentiable Simulation

The differentiable contact features with respect to object pose for our abstracted DSFs can be utilized in the differentiable simulations. In [33], the derivatives of contact features with respect to joint positions are computed to smooth the forward contact dynamics and solve a manipulation planning problem: finding a joint position trajectory for an initial and final pose of an object. The implementation in [33] relies on automatic differentiation and simple

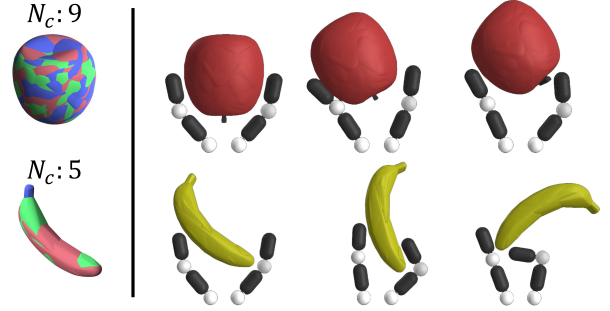


Figure 11. The results of manipulation planning. The apple and the banana in YCB dataset [3] are abstracted by our DSF abstraction method into 9 and 5 DSFs (N_c), respectively. They are imported in simulation and the iMPC in [33] solves the optimal joint position trajectory for a given initial and final pose of objects.

collision proxies including sphere and box. We integrate our abstracted DSFs into [33] and employ the differentiable contact features from the contact feature solver [19].

The results of the manipulation planning problem are illustrated in Fig. 11. The object is abstracted into DSFs by Alg. 3 and imported into the simulation. Then, the iterative MPC (iMPC) framework in [33] solves the manipulation planning problem by utilizing our analytic gradients.

7. Discussions and Conclusions

We propose two techniques for DSF-based shape abstraction: the differentiable shape parameterization and the hyperplane-based marching. The differentiable shape parameterization enables the efficient and robust DSF fitting across a wide range of convex geometries, providing differentiable contact features with respect to the shape parameters. For complex geometries, the hyperplane-based marching method improves abstraction accuracy and reduces the required number of DSF components. Our method outperforms the state-of-the-art in terms of accuracy and efficiency, achieving higher accuracy with fewer components. Additionally, we provide three applications that leverage the differentiable contact features of abstracted DSFs: shape editing, contact localization, and differentiable simulation.

Although our current approach relies on the SDF data, additional visual information such as texture embeddings and differentiable rendering can be utilized through the differentiable formulations of DSF. This integration has the potential to enhance both accuracy and applicability across a range of computer vision tasks in future works.

Acknowledgements. This research was supported by the Ministry of Trade, Industry & Energy (MOTIE) of Korea (RS-2024-00419641), and the National Research Foundation (NRF) grant funded by the Ministry of Science and ICT (MSIT) of Korea (RS-2022-00144468).

References

- [1] S. An, S. Lee, J. Lee, S. Park, and D. Lee. Collision detection between smooth convex bodies via riemannian optimization framework. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12464–12471. IEEE, 2024. 2, 5, 6
- [2] I. Biederman. Recognition-by-components: a theory of human image understanding. *Psychological Review*, 94(2):115, 1987. 1
- [3] B. Calli, A. Singh, A. Walsman, S. Srinivasa, P. Abbeel, and A. M. Dollar. The ycb object and model set: Towards common benchmarks for manipulation research. In *2015 International Conference on Advanced Robotics (ICAR)*, pages 510–517. IEEE, 2015. 6, 8
- [4] Z. Chen and H. Zhang. Learning implicit fields for generative shape modeling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5939–5948, 2019. 2
- [5] Z. Chen, A. Tagliasacchi, and H. Zhang. Bsp-net: Generating compact meshes via binary space partitioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 45–54, 2020. 1, 2
- [6] B. Curless and M. Levoy. A volumetric method for building complex models from range images. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, pages 303–312, 1996. 2, 6
- [7] R. Deits and R. Tedrake. Computing large convex regions of obstacle-free space through semidefinite programming. In *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*, pages 109–124. Springer, 2015. 4
- [8] B. Deng, K. Genova, S. Yazdani, S. Bouaziz, G. Hinton, and A. Tagliasacchi. Cvxnet: Learnable convex decomposition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 31–44, 2020. 1, 2, 6, 7
- [9] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem. Brax—a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021. 2
- [10] M. Geilinger, D. Hahn, J. Zehnder, M. Bäcker, B. Thomaszewski, and S. Coros. Add: Analytically differentiable dynamics for multi-body systems with frictional contact. *ACM Transactions on Graphics (TOG)*, 39(6):1–15, 2020. 2
- [11] K. Genova, F. Cole, D. Vlasic, A. Sarna, W. T. Freeman, and T. Funkhouser. Learning shape templates with structured implicit functions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7154–7164, 2019. 2
- [12] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi. A fast procedure for computing the distance between complex objects in three-dimensional space. *IEEE Journal on Robotics and Automation*, 4(2):193–203, 1988. 2
- [13] T. Groueix, M. Fisher, V. G. Kim, B. C. Russell, and M. Aubry. A papier-mâché approach to learning 3d surface generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 216–224, 2018. 2
- [14] T. A. Howell, S. Le Cleac’h, J. Z. Kolter, M. Schwager, and Z. Manchester. Dojo: A differentiable simulator for robotics. *arXiv preprint arXiv:2203.00806*, 9, 2022. 2
- [15] M. Iskandar, O. Eiberger, A. Albu-Schäffer, A. De Luca, and A. Dietrich. Collision detection, identification, and localization on the dlr sara robot with sensing redundancy. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3111–3117. IEEE, 2021. 7
- [16] M. Iskandar, A. Albu-Schäffer, and A. Dietrich. Intrinsic sense of touch for intuitive physical human-robot interaction. *Science Robotics*, 9(93):eadn4008, 2024. 7
- [17] C. Kong, C. H. Lin, and S. Lucey. Using locally corresponding cad models for dense 3d reconstructions from a single image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4857–4865, 2017. 2
- [18] S. Le Cleac’h, H. X. Yu, M. Guo, T. Howell, R. Gao, J. Wu, Z. Manchester, and M. Schwager. Differentiable physics simulation of dynamics-augmented neural objects. *IEEE Robotics and Automation Letters*, 8(5):2780–2787, 2023. 2
- [19] J. Lee, M. Lee, and D. Lee. Uncertain pose estimation during contact tasks using differentiable contact features. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 2, 8
- [20] M. Liu, R. Shi, L. Chen, Z. Zhang, C. Xu, X. Wei, H. Chen, C. Zeng, J. Gu, and H. Su. One-2-3-45++: Fast single image to 3d objects with consistent multi-view generation and 3d diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10072–10083, 2024. 2, 4
- [21] M. Liu, C. Xu, H. Jin, L. Chen, M. Varma T, Z. Xu, and H. Su. One-2-3-45: Any single image to 3d mesh in 45 seconds without per-shape optimization. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2024. 2, 4
- [22] W. Liu, Y. Wu, S. Ruan, and G. S. Chirikjian. Robust and accurate superquadric recovery: A probabilistic approach. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2676–2685, 2022. 1, 2, 6
- [23] W. Liu, Y. Wu, S. Ruan, and G. S. Chirikjian. Marching-primitives: Shape abstraction from signed distance function. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8771–8780, 2023. 1, 2, 4, 6, 7
- [24] W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. *ACM SIGGRAPH Computer Graphics*, 21(4):163–169, 1987. 4
- [25] K. Mamou, E. Lengyel, and A. Peters. Volumetric hierarchical approximate convex decomposition. *Game Engine Gems*, 3:141–158, 2016. 2, 4
- [26] J. McCrae, K. Singh, and N. J. Mitra. Slices: a shape-proxy based on planar sections. *ACM Transactions on Graphics (TOG)*, 30(6):168, 2011. 1
- [27] J. McCrae, N. J. Mitra, and K. Singh. Surface perception of planar abstractions. *ACM Transactions on Applied Perception (TAP)*, 10(3):1–20, 2013. 1

- [28] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger. Occupancy networks: Learning 3d reconstruction in function space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4460–4470, 2019. 2
- [29] T. Monnier, J. Austin, A. Kanazawa, A. Efros, and M. Aubry. Differentiable blocks world: Qualitative 3d decomposition by rendering primitives. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5791–5807, 2023. 1, 2
- [30] L. Montaut, Q. Le Lidec, A. Bambade, V. Petrik, J. Sivic, and J. Carpentier. Differentiable collision detection: a randomized smoothing approach. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3240–3246. IEEE, 2023. 2
- [31] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto. Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1366–1373. IEEE, 2017. 2
- [32] J. Ortiz, A. Clegg, J. Dong, E. Sucar, D. Novotny, M. Zollhoefer, and M. Mukadam. isdf: Real-time neural signed distance fields for robot perception. In *Proceedings of Robotics: Science and Systems*, 2022. 2, 4
- [33] T. Pang, H. J. T. Suh, L. Yang, and R. Tedrake. Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models. *IEEE Transactions on Robotics (TRO)*, 2023. 2, 8
- [34] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove. Deepsdf: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 165–174, 2019. 2, 4
- [35] D. Paschalidou, A. O. Ulusoy, and A. Geiger. Superquadrics revisited: Learning 3d shape parsing beyond cuboids. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10344–10353, 2019. 1, 2
- [36] K. Pearson. LIII. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901. 4
- [37] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 652–660, 2017. 1
- [38] C. Y. Sun, Q. F. Zou, X. Tong, and Y. Liu. Learning adaptive hierarchical cuboid abstractions of 3d shape collections. *ACM Transactions on Graphics (TOG)*, 38(6):1–13, 2019. 1, 2
- [39] K. Tracy, T. A. Howell, and Z. Manchester. Differentiable collision detection for a set of convex primitives. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3663–3670. IEEE, 2023. 2
- [40] S. Tulsiani, H. Su, L. J. Guibas, A. A. Efros, and J. Malik. Learning shape abstractions by assembling volumetric primitives. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2635–2643, 2017. 1, 2
- [41] D. Turpin, L. Wang, E. Heiden, Y. C. Chen, M. Macklin, S. Tsogkas, S. Dickinson, and A. Garg. Grasp’d: Differentiable contact-rich grasp synthesis for multi-fingered hands. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 201–221. Springer, 2022. 2
- [42] D. Turpin, T. Zhong, S. Zhang, G. Zhu, E. Heiden, M. Macklin, S. Tsogkas, S. Dickinson, and A. Garg. Fast-grasp’d: Dexterous multi-finger grasp generation through differentiable simulation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8082–8089. IEEE, 2023. 2
- [43] G. Van Den Bergen. Proximity queries and penetration depth computation on 3d game objects. In *Game Developers Conference*, page 209, 2001. 2
- [44] N. Vaskevicius and A. Birk. Revisiting superquadric fitting: A numerically stable formulation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(1):220–233, 2017. 2
- [45] N. Wang, Y. Zhang, Z. Li, Y. Fu, W. Liu, and Y. G. Jiang. Pixel2mesh: Generating 3d mesh models from single rgb images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 52–67, 2018. 2
- [46] P. Wang, L. Liu, Y. Liu, C. Theobalt, T. Komura, and W. Wang. Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 1
- [47] X. Wei, M. Liu, Z. Ling, and H. Su. Approximate convex decomposition for 3d meshes with collision-aware concavity and tree search. *ACM Transactions on Graphics (TOG)*, 41(4):1–18, 2022. 2, 4, 6, 7
- [48] K. Yang and X. Chen. Unsupervised learning for cuboid shape abstraction via joint segmentation from point clouds. *ACM Transactions on Graphics (TOG)*, 40(4):1–11, 2021. 1, 2
- [49] H. Yuan, J. Xu, H. Pan, A. Bousseau, N. J. Mitra, and C. Li. Cadtalk: An algorithm and benchmark for semantic commenting of cad programs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3753–3762, 2024. 1
- [50] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo, 2022. 6
- [51] D. Zurlo, T. Heitmman, M. Morlock, and A. De Luca. Collision detection and contact point estimation using virtual joint torque sensing applied to a cobot. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7533–7539. IEEE, 2023. 7